



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet 2

JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : Java avancé



Chapitre

4

Les interfaces graphiques avec Swing

Sofiane HACHICHA
ISIMG 2025 - 2026

4- Swing

4.1 Introduction



- **La bibliothèque Java contient trois ensembles de classes permettant la création d'interfaces utilisateurs graphiques. Dans l'ordre chronologique de leur apparition, il s'agit de :**
 - AWT (Abstract Window Toolkit), ce n'est plus d'actualité mais servant de base à Swing,
 - Swing, utilisé pour la plupart des applications jusqu'à récemment,
 - Java FX.
- **Ce chapitre a pour objectif de présenter Swing, mais la plupart des concepts se retrouvent dans Java FX et dans d'autres bibliothèques similaires pour d'autres langages.**
- **Swing est composé d'un très grand nombre de classes appartenant toutes (ou presque) au paquetage `javax.swing` et à ses sous-paquetages.**
 - Etant donné que Swing se base sur AWT, les classes du paquetage `java.awt` et ses sous-paquetages sont souvent nécessaires.





- Une interface graphique Swing se construit en combinant un certain nombre de composants (components), imbriqués les uns dans les autres.
 - Par exemple : les fenêtres, les menus, les boutons, les zones de texte, etc. sont des composants.
- Les classes représentant ces composants ont généralement un nom commençant par **J**, suivi du type du composant (par exemple JButton) et héritent de la classe abstraite **javax.swing.JComponent**.
- Il existe deux types de composants Swing :
 - les **composants de base**, qui ne contiennent pas d'autres composants (boutons, etc.),
 - les **conteneurs**, dont le but principal est de contenir et d'organiser d'autres composants (fenêtres, etc.).



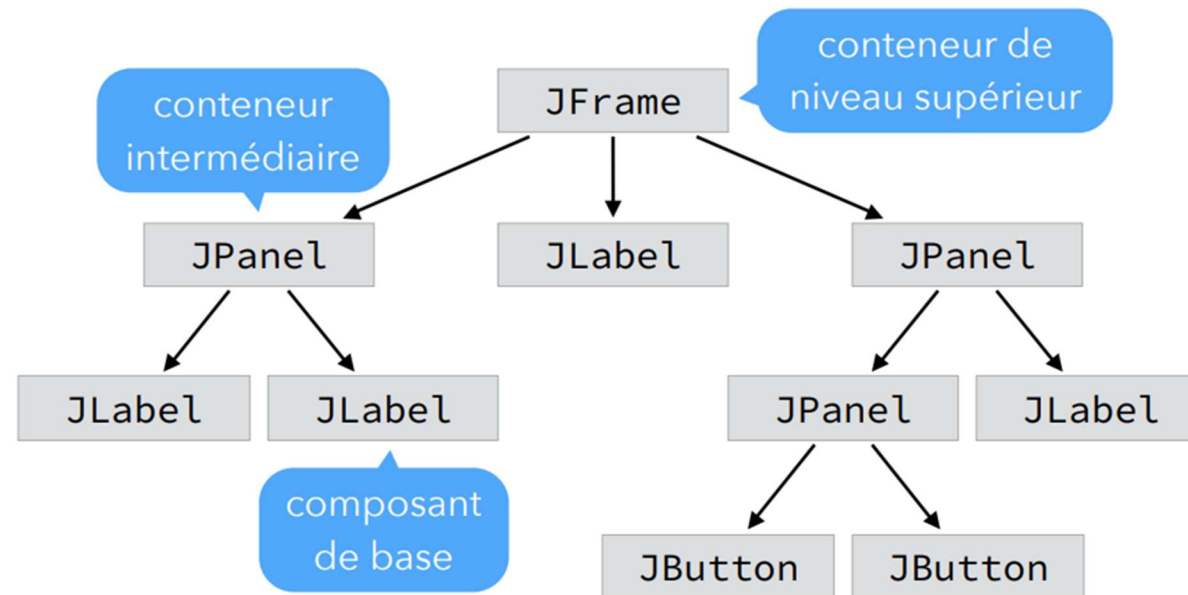


- **Les conteneurs sont séparés en deux catégories :**
 - les **conteneurs de niveau intermédiaire**, qui peuvent eux-mêmes être contenus par d'autres conteneurs,
 - les **conteneurs de niveau supérieur** (top-level containers), qui ne peuvent pas l'être.
- **Les composants d'une application Swing sont organisés en une hiérarchie arborescente dont :**
 - la **racine** est un conteneur de niveau supérieur,
 - les **nœuds** - autres que la racine - sont des composants intermédiaires,
 - les **feuilles** sont des composants de base.
- **Le principal type de conteneur de niveau supérieur est la fenêtre (classe **JFrame**), donc à chaque fenêtre affichée à l'écran, par une application Swing, correspond une hiérarchie.**



4- Swing

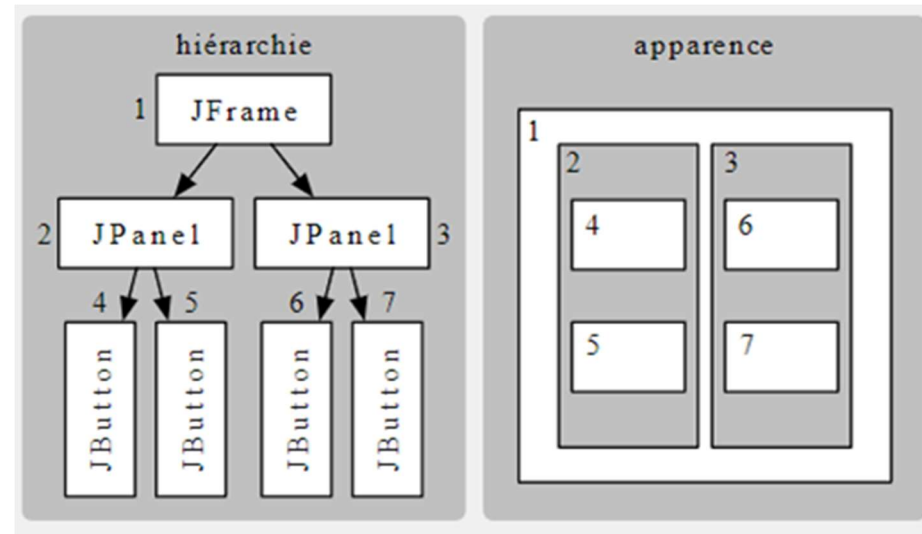
4.2 Composants



● La figure ci-dessus présente un exemple (simplifié) de hiérarchie correspondant à une application Swing constituée d'une fenêtre contenant plusieurs composants.

4- Swing

4.2 Composants



- A l'écran, tout composant occupe une zone rectangulaire. Les fils d'un conteneur se partagent généralement sa zone à lui, et il n'y a pas de chevauchement entre composants frères.
- La hiérarchie des composants est donc aplatie en un ensemble de zones rectangulaires imbriquées.



- La classe abstraite **JComponent** sert de classe mère à tous les composants Swing, sauf ceux de niveau supérieur.
- JComponent et ses sous-classes possèdent un grand nombre de **propriétés** (attributs) modifiables.
 - A chacune d'entre-elles est associée une méthode de lecture **getXxx** et une méthode d'écriture **setXxx** où **Xxx** représente le nom de la propriété.
 - Par exemple, la propriété **font**, qui contient la police utilisée pour le texte d'un composant, se lit avec la méthode **getFont** et s'écrit avec la méthode **setFont**.
- Les composants de base offerts par Swing incluent :
 - une étiquette, textuelle ou graphique (JLabel),
 - différents boutons : à un état (JButton), à deux états (JToggleButton), « radio » (JRadioButton), à cocher (JCheckBox),
 - une liste de valeurs sélectionnables (JList),



4- Swing

4.2.1 Composants Swing



- un champ textuel (JTextField), également en version formatante (JFormattedTextField), ou masquée pour les mots de passe (JPasswordField),
- etc.
- **Les conteneurs intermédiaires offerts par Swing incluent :**
 - un panneau séparé en 2 parties redimensionnables, affichant chacune un composant (JSplitPane),
 - un panneau à onglets (JTabbedPane),
 - un panneau affichant une partie d'un composant trop grand pour être affiché en entier (JScrollPane),
 - un panneau permettant de superposer plusieurs composants (JLayeredPane),
 - un panneau sans représentation graphique (JPanel),
 - etc.

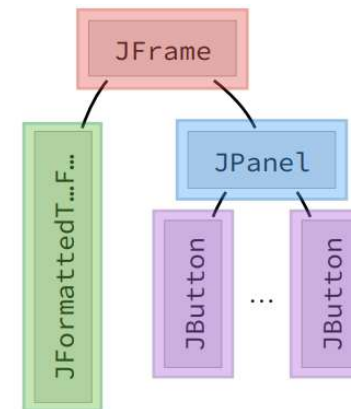
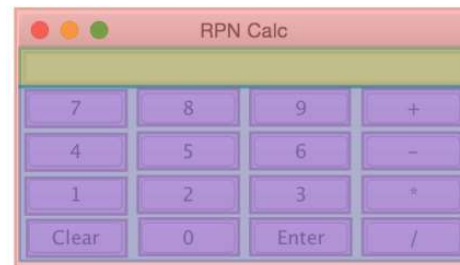




● Les conteneurs de niveau supérieur offerts par Swing sont au nombre de trois :

- la fenêtre (JFrame),
- la boîte de dialogue (JDialog),
- l'« applet » (JApplet) destinée principalement à être utilisée dans un navigateur Web, dépréciée depuis Java 9.

● Par exemple, l'interface graphique d'une calculatrice pourrait être obtenue au moyen de la hiérarchie suivante :





4.2.2 Composants de niveau supérieur



- **Les composants de niveau supérieur sont à la racine de toute hiérarchie visible à l'écran.**
 - Contrairement aux autres types de composants, ceux de niveau supérieur ne peuvent pas être inclus dans d'autres composants.
- **Swing offre trois types de conteneurs de niveau supérieur: la fenêtre, la boîte de dialogue et l'applet.**





● La classe **JFrame** est utilisée pour représenter une fenêtre graphique.

- Ses principales **propriétés** sont : son titre (**title**), sa taille et sa position à l'écran (**bounds**), sa visibilité (**visible**, valant **false** par défaut), son comportement en cas de fermeture (**defaultCloseOperation**) et son panneau de contenu (**contentPane**).



- Par défaut, lorsque l'utilisateur ferme une fenêtre dans une application Java Swing en cliquant sur le bouton de fermeture, la fenêtre est simplement rendue **invisible** mais l'application continue de s'exécuter en arrière-plan.





- Cependant, pour la fenêtre principale d'une application, il est souhaitable que la fermeture de cette fenêtre entraîne également la fin de l'application.

- Pour que la fermeture de la fenêtre principale entraîne la sortie de l'application, il suffit de modifier la propriété **defaultCloseOperation** en lui attribuant la valeur **EXIT_ON_CLOSE**

```
JFrame f = new JFrame("Ma fenêtre principale");  
f.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE);
```

- L'agencement des composants à l'écran se réalise de manière récursive, en commençant par les feuilles puis en remontant jusqu'à la racine.
- La classe JFrame, héritant de java.awt.Window, propose la méthode **pack()** pour ajuster automatiquement la taille minimale de la fenêtre principale en fonction de ses composants, ainsi que la méthode **dispose()** pour fermer la fenêtre principale et libérer les ressources qui lui sont allouées.





4- Swing

4.2.2.1 JFrame



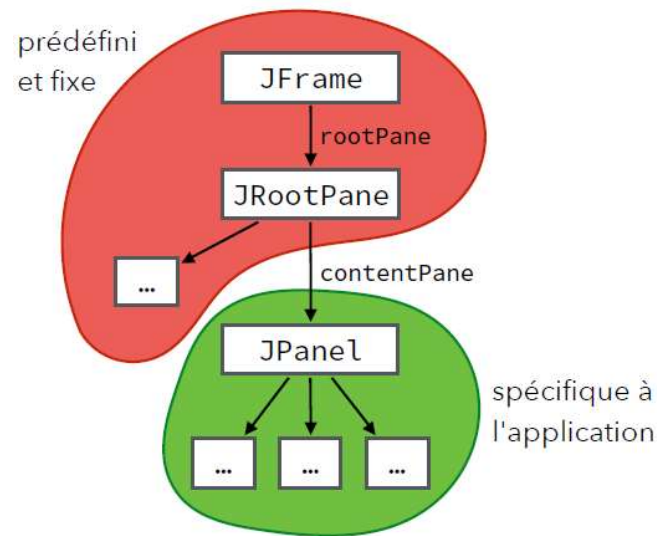
- La classe **JFrame**, héritant de **java.awt.Window**, propose la méthode **setLocationRelativeTo(Component c)** utilisée pour positionner la fenêtre principale par rapport à un composant spécifique ou par rapport à l'écran. Elle prend en paramètre un composant (tel qu'un autre conteneur Swing) ou la valeur **null** pour centrer la fenêtre par rapport à l'écran.





- La classe **JDialog** est utilisée pour représenter une boîte de dialogue, qui est visuellement similaire à une fenêtre mais se comportant différemment.
 - Une boîte de dialogue est toujours associée à une fenêtre existante et dépend de celle-ci, bien qu'elle ne soit pas contenue à l'intérieur de la fenêtre.
- De nombreuses boîtes de dialogue sont **modales**, ce qui signifie que toute interaction avec le reste de l'application est bloquée tant que la boîte de dialogue n'est pas fermée, comme c'est le cas avec **JOptionPane**. Pour créer des boîtes de dialogue **non modales** qui n'interfèrent pas avec le reste de l'application, on doit utiliser la classe **JDialog**.
- La classe **JDialog** offre la possibilité de créer des boîtes de dialogue entièrement personnalisées selon les besoins de l'application.

- Les conteneurs de niveau supérieur ne contiennent pas directement un certain nombre de composants fils.
- Au lieu de cela, ils possèdent un unique **panneau racine** (rootPane), un type spécial de conteneur intermédiaire.
- Ce panneau racine contient, entre autres, un **panneau de contenu** (contentPane), dans lequel les composants spécifiques à l'application seront ajoutés.





4.2.2.3 Panneaux racine/contenu



- La plupart des applications ignorent le panneau racine et interagissent uniquement avec le panneau de contenu.
- Le panneau de contenu est une propriété modifiable des conteneurs de niveau supérieur, appelée **contentPane**.





4.2.3 Conteneurs intermédiaires



- Les conteneurs intermédiaires sont les nœuds - autres que la racine - de la hiérarchie.
- Le but principal d'un conteneur intermédiaire est de grouper et d'organiser un certain nombre d'autres composants, nommé ses **fils** . Dès lors, sa représentation graphique propre est souvent minimale, voire inexistante.





- **JSplitPane permet de diviser le composant en deux parties, chacune affichant un composant fils.**
 - La division peut être verticale ou horizontale, et est redimensionnable, éventuellement par l'utilisateur.



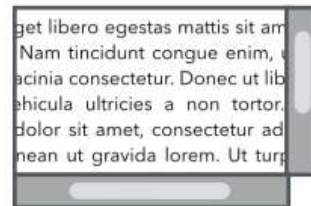


● **JTabbedPane est un panneau composé d'un certain nombre d'onglets affichant chacun un composant fils différent.**

- Un seul onglet est visible à un instant donné



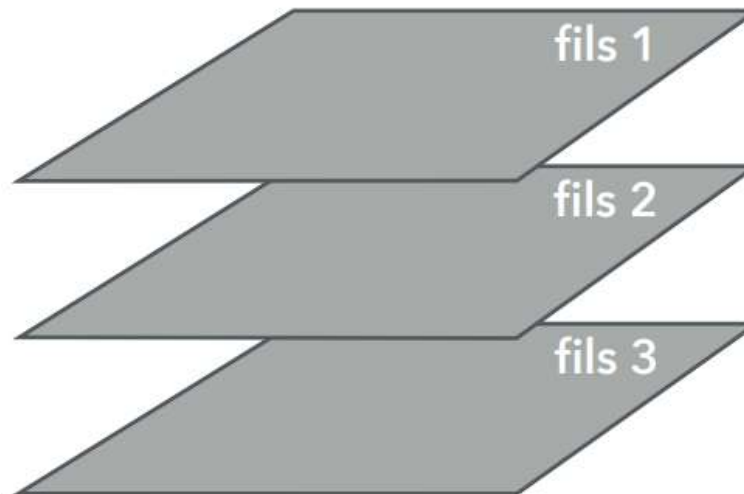
- **JScrollPane donne accès à une sous-partie d'un composant trop grand pour tenir à l'écran et permet de déplacer la zone visualisée de différentes manières, p.ex. au moyen de barres de défilement.**



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor. Pellentesque auctor nisi id magna consequat sagittis. Curabitur dapibus enim sit amet elit pharetra tincidunt feugiat nisl imperdiet. Ut convallis libero in urna ultrices accumsan. Donec sed odio eros. Donec viverra mi quis quam pulvinar at malesuada arcu rhoncus. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. In rutrum accumsan ultricies. Mauris vitae nisi at sem facilisis semper ac in est.



- **JLayeredPane permet de superposer plusieurs composants, ce qui peut être utile pour dessiner au-dessus de composants existants ou pour intercepter les clics de souris qui leur sont destinés.**



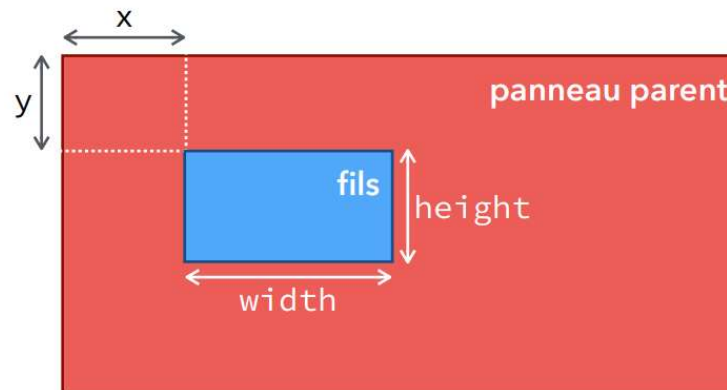


- **JPanel représente un panneau, un conteneur intermédiaire sans représentation graphique propre, à l'exception éventuelle de sa bordure. Il peut contenir un nombre variable de composants fils. Le rôle principal d'un JPanel est d'organiser les interfaces en regroupant et positionnant d'autres composants via son gestionnaire d'agencement.**
- **Les méthodes **add** et **remove** permettent de gérer les fils d'un conteneur JPanel :**
 - `void add(Component c, Object obj, int index)` : insère le composant **c** à la position **index** dans les fils du panneau (0 pour la première position, -1 pour la fin) et lui associe des informations d'agencement via **obj**.
 - `void add(Component c, Object obj)` équivalent à `add(c, obj, -1)`
 - `Component add(Component c, int index)` équivalent à `add(c, null, index)`





- Component add(Component c) équivalent à add(c, null).
- void remove(Component c) : supprime le composant c des fils du panneau.
- void remove(int index) : supprime le composant à la position index.
- **Chaque composant fils d'un panneau occupe un rectangle à l'intérieur de celui de son parent, appelé bornes (bounds), qui comprend la position (x, y) et la taille (width, height), exprimées dans le repère du panneau.**





- **L'agencement des fils - leur positionnement à l'intérieur du rectangle de leur parent - peut être effectué de deux manières :**
 1. « manuellement », en modifiant leurs bornes au moyen de la méthode `setBounds` :
`setBounds(int x, int y, int width, int height)`
 2. via un gestionnaire d'agencement (**layout manager**) attaché au parent et responsable de l'agencement de ses fils et de son dimensionnement.
- **En général, la deuxième manière est préférée dans la plupart des cas.**



4.2.3.6 Gestionnaires d'agencement



- **Les gestionnaires d'agencement peuvent utiliser plusieurs caractéristiques des composants fils pour les placer :**
 - leur index dans la séquence des fils,
 - leur taille minimale (propriété `minimumSize`), préférée (`preferredSize`) et maximale (`maximumSize`)
 - l'éventuelle information d'agencement qui leur a été associée au moment de leur ajout via la méthode **add**.
- **Attention : les gestionnaires ont le droit mais pas l'obligation d'utiliser ces caractéristiques. En particulier, plusieurs gestionnaires ignorent les informations de taille.**
- **L'interface `LayoutManager` représente un gestionnaire d'agencement. Swing fournit plusieurs gestionnaires d'agencement prédéfinis qui implémentent cette interface. Parmi ces gestionnaires, on retrouve : `FlowLayout`, `BorderLayout`, `GridLayout`, `GridBagLayout`, `BoxLayout`, etc.**
 - Ces gestionnaires agencent chacun des composants fils en fonction d'une technique qui leur est propre.

4.2.3.6 Gestionnaires d'agencement



- **Tout conteneur possède un gestionnaire d'agencement par défaut.**
 - Les gestionnaires par défaut sont **FlowLayout** pour JPanel et **BorderLayout** pour JFrame (ContentPane), JDialog et JApplet.
- **Une fois installé, le gestionnaire d'agencement fonctionne "tout seul" en interagissant avec le conteneur.**
- **Pour affecter ou changer le gestionnaire d'agencement d'un conteneur, il est possible d'utiliser :**

```
conteneur.setLayout(new XYZLayout())
```
- **JPanel possède un constructeur bien pratique qui prend un gestionnaire d'agencement comme argument :**

```
JPanel p = new JPanel(new XYZLayout());
```
- **On peut imposer à un conteneur de ne pas avoir de gestionnaire d'agencement :**

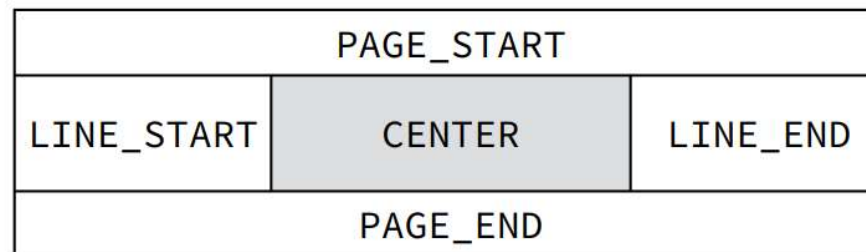
```
conteneur.setLayout(null);
```

 - Le programmeur doit ainsi positionner chacun des composants « manuellement »





- BorderLayout découpe le conteneur en **cinq régions** (ou zones) **nommées**, chacune d'entre-elles pouvant contenir au plus un composant.



- Toutes les régions, à l'exception de la région centrale, sont dimensionnées en fonction de leur contenu - qui peut être inexistant.
 - Si une région n'a pas de contenu ajouté, elle aura une taille zéro dans cette direction.
 - la totalité de l'espace restant dans le conteneur est attribuée à la région centrale.



- **FlowLayout positionne les composants fils de la même manière qu'ils sont arrangés en lignes successives.**
 - **Direction de l'agencement** : On peut spécifier la direction de l'agencement en utilisant `ComponentOrientation.LEFT_TO_RIGHT` (qui est la valeur par défaut) pour un agencement de gauche à droite, ou `ComponentOrientation.RIGHT_TO_LEFT` pour un agencement de droite à gauche.
 - **Alignement des lignes** : Les lignes peuvent être alignées à gauche, à droite ou centrées en utilisant les constantes `FlowLayout.LEFT`, `FlowLayout.RIGHT`, `FlowLayout.CENTER` (valeur par défaut), `FlowLayout.LEADING` et `FlowLayout.TRAILING`.
 - **Espacement** : On peut définir l'espacement horizontal et vertical entre les composants en utilisant les méthodes `setHgap(int hgap)` et `setVgap(int vgap)` du `FlowLayout`. Par défaut, l'espacement horizontal et vertical est de 5 pixels.

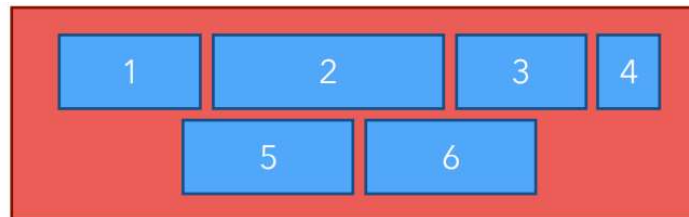




- **Les constantes FlowLayout.LEADING et FlowLayout.TRAILING sont utilisées pour aligner les composants dans un FlowLayout selon la direction de l'agencement (ComponentOrientation).**
 - FlowLayout.LEADING aligne les composants **du côté de départ de l'agencement**, par exemple à gauche dans une orientation de gauche à droite (LEFT_TO_RIGHT) et à droite dans une orientation de droite à gauche (RIGHT_TO_LEFT).
 - FlowLayout.TRAILING aligne les composants **du côté opposé au départ de l'agencement**, par exemple à droite dans une orientation de gauche à droite (LEFT_TO_RIGHT) et à gauche dans une orientation de droite à gauche (RIGHT_TO_LEFT).
- **Ces constantes permettent d'adapter dynamiquement l'alignement des composants en fonction de la direction de l'agencement, ce qui est particulièrement utile dans des interfaces utilisateur multilingues ou bidirectionnelles.**



- Par exemple, on peut agencer six composants fils d'un panneau en les plaçant de gauche à droite et en centrant chaque ligne.



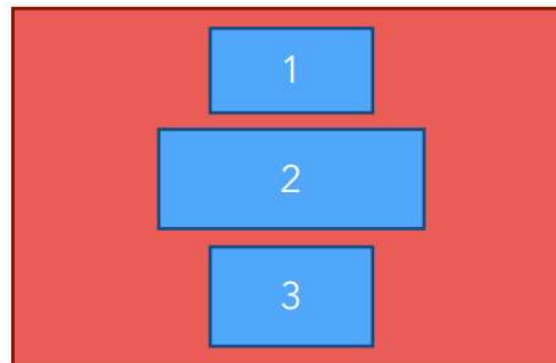
- Voici un exemple d'utilisation du FlowLayout avec des configurations spécifiques :

```
JPanel p = new JPanel(new FlowLayout(FlowLayout.TRAILING, 10, 20));
```

- Dans cet exemple, un JPanel est créé avec un FlowLayout qui dispose les composants de gauche à droite (par défaut), les aligne à droite (opposé à l'orientation par défaut), et sépare chaque composant par un espacement de 10 pixels horizontalement et de 20 pixels verticalement.

● **BoxLayout dispose les composants fils dans le conteneur selon deux orientations possibles : horizontale ou verticale.**

- Dans une orientation horizontale, les composants sont alignés côte à côte de gauche à droite.
- Dans une orientation verticale, les composants sont empilés les uns au-dessus des autres de haut en bas.



● **La façon dont les composants sont insérés dans le conteneur dépend du deuxième paramètre passé au constructeur BorderLayout qui peut être :**



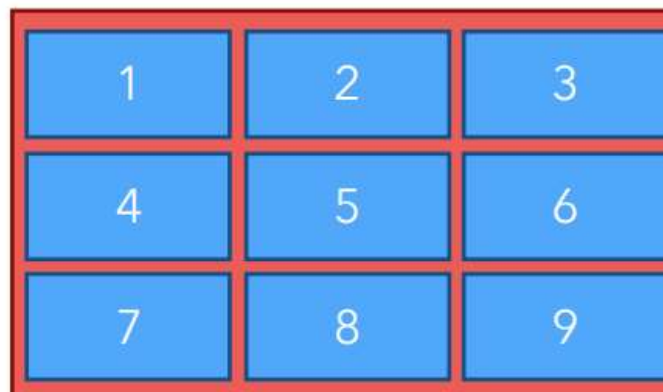
- **X_AXIS** : Les composants fils sont placés horizontalement de gauche à droite, indépendamment de l'orientation du conteneur.
 - **Y_AXIS** : Les composants fils sont placés verticalement de haut en bas, indépendamment de l'orientation du conteneur.
 - **LINE_AXIS** : Comme X_AXIS, mais en respectant l'orientation du conteneur, ce sera éventuellement de droite à gauche.
 - **PAGE_AXIS** : Comme Y_AXIS, mais en respectant l'orientation du conteneur.
- **Voici un exemple d'utilisation du BorderLayout :**
- ```
JPanel panel = new JPanel();
panel.setComponentOrientation(ComponentOrientation.RIGHT_TO_LEFT);
panel.setLayout(new BorderLayout(panel, BorderLayout.LINE_AXIS));
```
- Dans ce code, un JPanel est créé avec un BorderLayout orienté horizontalement de droite à gauche.



- **GridLayout dispose les composants fils dans une grille de  $n \times m$  cellules de taille identique. Le placement commence généralement au coin supérieur gauche de la grille et suit le sens de lecture, c'est-à-dire de gauche à droite puis de haut en bas.**

- Par exemple, on peut agencer neuf composants fils d'un panneau dans une grille de  $3 \times 3$  cellules

```
JPanel panel = new JPanel(new GridLayout(3, 3));
```





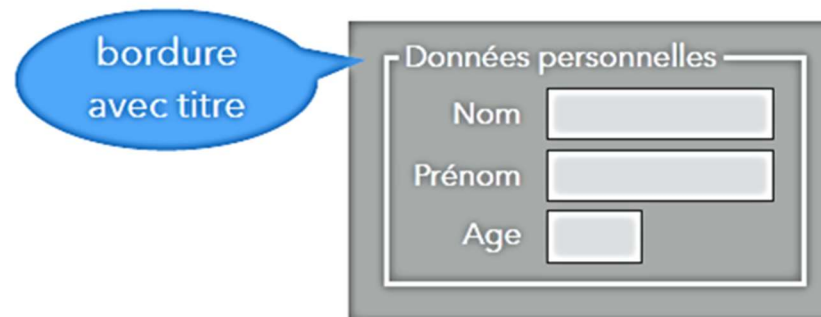
● **Swing offre encore d'autres gestionnaires d'agencement, plus puissants que ceux présentés jusqu'ici, parmi lesquels :**

- **GridBagLayout**, version améliorée de GridLayout permettant de dimensionner les lignes et colonnes individuellement, et de combiner des cellules,
- **GroupLayout**, associant chaque composant à un groupe horizontal et un groupe vertical et alignant de manière configurable tous les composants d'un groupe.





- Il est possible d'ajouter une bordure (**border**) à n'importe quel composant au moyen de la méthode **setBorder** de **JComponent**.
- Il est assez fréquent d'ajouter une bordure à un panneau (**JPanel**) afin de grouper visuellement un certain nombre de composants liés logiquement.
  - Par exemple, une bordure peut être utilisée pour entourer et nommer les données personnelles dans un formulaire, comme dans la figure suivante :



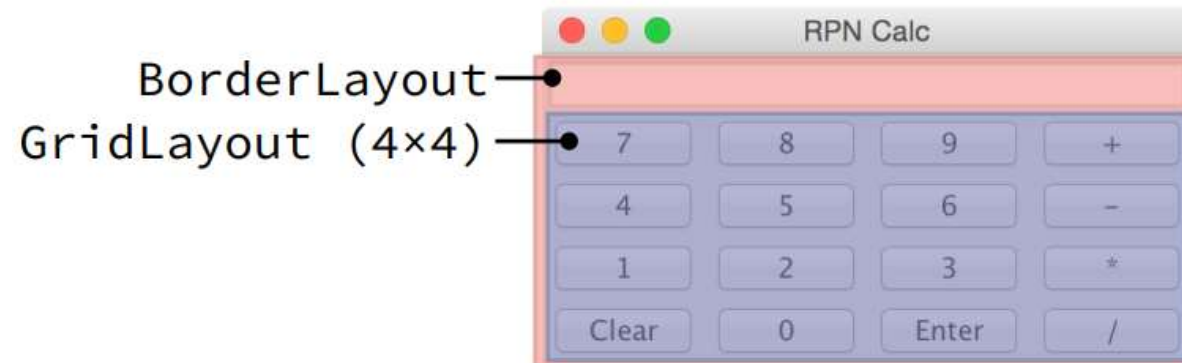


- Il est recommandé de ne pas créer directement les bordures en utilisant l'opérateur new, mais plutôt d'utiliser les méthodes fabriques statiques de la classe **BorderFactory** pour créer différents types de bordures :
  - **createEmptyBorder(int top, int left, int bottom, int right)** : Crée une bordure vide autour d'un composant graphique en spécifiant les dimensions des marges supérieure, gauche, inférieure et droite de la bordure vide à créer.
  - **createLineBorder(Color color)** : Crée une bordure composée d'une simple ligne avec la couleur spécifiée. Il est possible de spécifier l'épaisseur de la ligne.
  - **createTitledBorder(Border border, String title)** : Prend une bordure existante et/ou une chaîne, qu'elle lui ajoute en titre.
  - etc.





- L'interface de la calculatrice , déjà présentée, peut être mise en page au moyen de deux gestionnaires :
  - le premier de type **BorderLayout**, attaché au panneau de contenu, permettant de placer l'affichage dans la zone PAGE\_START et le panneau du clavier dans la zone CENTER,
  - le second de type **GridLayout** et de taille 4×4, attaché au panneau clavier.





#### Le code correspondant est le suivant :

```
JFrame frame = new JFrame("RPN Calc");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
JPanel p = new JPanel(new BorderLayout());
p.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
JFormattedTextField affichage = new JFormattedTextField(); affichage.setValue(null);
p.add(affichage, BorderLayout.PAGE_START);
JPanel clavier = new JPanel (new GridLayout(4, 4, 10, 10));
clavier.add(new Button("7")); clavier.add(new Button("8"));
clavier.add(new Button("9")); clavier.add(new Button("+"));
clavier.add(new Button("4")); clavier.add(new Button("5"));
clavier.add(new Button("6")); clavier.add(new Button("-"));
clavier.add(new Button("1")); clavier.add(new Button("2"));
clavier.add(new Button("3")); clavier.add(new Button("*"));
clavier.add(new Button("Clear")); clavier.add(new Button("0"));
clavier.add(new Button("Enter")); clavier.add(new Button("/"));
p.add(clavier, BorderLayout.CENTER);
frame.getContentPane().add(p);
frame.pack(); frame.setLocationRelativeTo(null);
frame.setVisible(true);
```





- Les composants de base sont ceux apparaissant aux feuilles de la hiérarchie. Ils ne peuvent pas contenir d'autres composants.







● **JLabel** représente l'un des composants les plus simples qui soit, une étiquette textuelle et/ou graphique.

- Ses principales propriétés sont le texte (**text**) et l'image (**icon**), au format GIF, qu'elle affiche (l'image et le texte sont alignés ensemble horizontalement), pouvant les deux être vides (null).

- **L'alignement vertical** par défaut d'un JLabel est centré. Pour modifier cet alignement, il suffit d'utiliser les constantes : `SwingConstants.TOP` et `SwingConstants.BOTTOM`.

- **L'alignement horizontal** par défaut est soit à droite s'il ne contient que du texte, soit centré s'il contient une image avec ou sans texte. Pour modifier cet alignement, il suffit d'utiliser les constantes : `SwingConstants.LEFT`, `SwingConstants.CENTER`, et `SwingConstants.RIGHT`.



### 4.2.4.2 Composants de base : Bouton à un état



- **JButton** représente un bouton à un état, destiné à être cliqué pour effectuer une action.
  - Sa principale propriété est le texte qu'il affiche (**text**).
- **Des auditeurs peuvent lui être attachés pour gérer les clics de l'utilisateur - voir plus loin**



### 4.2.4.3 Composants de base : Case à cocher



- **JCheckBox** représente une case à cocher, qui peut être sélectionnée ou non.
- Ses principales propriétés sont le texte qu'elle affiche (**text**) et son état (**selected**).





- **JRadioButton** représente un bouton « radio », similaire à une case à cocher mais faisant partie d'un groupe (de type **ButtonGroup**) dont un seul peut être sélectionné.
  - Ses principales propriétés sont les mêmes que celles de **JCheckBox**.
- **Un JRadioButton est un bouton qui peut être sélectionné ou non. Un ButtonGroup permet de grouper des boutons radio, de façon qu'il n'y ait qu'un seul bouton sélectionné dans le groupe.**





● **JComboBox permet de sélectionner une valeur parmi une liste prédéfinie. Ses principales fonctionnalités incluent :**

- Le modèle de données (**model**), par défaut un **DefaultComboBoxModel**, pour gérer les éléments.
- La possibilité de rendre la JComboBox éditable avec la méthode **setEditable(boolean editable)** (par défaut, JComboBox n'est pas éditable).
- La méthode **addItem(Object item)** ajoute un élément à la fin de la liste, tandis que la méthode **insertItemAt(Object item, int index)** insère un élément à une position spécifique dans la liste.
- Les méthodes **setSelectedItem(Object item)** et **setSelectedIndex(int index)** pour sélectionner un élément spécifique dans la liste.



### 4.2.4.6 Composants de base : Composants textuels



- **TextField** et ses sous-classes **FormattedTextField** et **PasswordField** sont des composants graphiques permettant la saisie de texte sur une seule ligne avec une présentation uniforme, y compris une police de caractères uniforme.

- Leur principale propriété est le texte (**text**), qui est représenté comme une chaîne de caractères de type String.



- **FormattedTextField** est capable de stocker une valeur non textuelle dans sa propriété **value**. Pour afficher cette valeur sous forme de texte et inversement, le **FormattedTextField** utilise un formateur qui lui est attaché

### 4.2.4.6 Composants de base : Composants textuels



- **JTextArea** permet la saisie du texte sur plusieurs lignes, tandis que **JTextPane** et **JEditorPane** sont capables d'afficher du texte mis en page et d'offrir des fonctionnalités d'édition supplémentaires.
  - Leur principale propriété est le texte qu'ils contiennent, accessible soit sous forme de texte brut (**text**) soit en version mise en page (**document**).
- **L'exemple suivant illustre l'utilisation d'un JTextArea**

Lorem ipsum dolor sit amet, consectetur adipiscing elit. *Quisque tincidunt dolor a convallis sagittis.* Vestibulum dapibus massa arcu, id cursus est sodales sit amet. Phasellus mi ex, **posuere** in nisi a, varius malesuada urna.

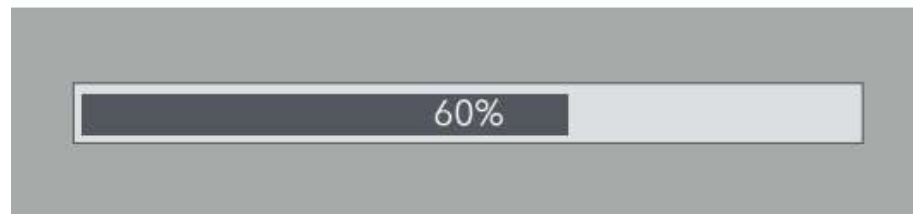


### 4.2.4.7 Composants de base : Barre de progression



● **JProgressBar** représente une barre de progression qui est utilisée pour visualiser l'état d'avancement d'une opération en cours, généralement une opération de longue durée.

- Sa principale propriété est sa valeur (**value**), qui un entier compris entre une valeur minimale (**minimum**) et une valeur maximale (**maximum**) définies, permettant ainsi de représenter l'état d'avancement actuel de manière visuelle.
- La méthode **setStringPainted(boolean)** permet de spécifier si la barre de progression doit afficher le pourcentage de progression sous forme de texte.







### 4.2.4.8 Composants de base : Macro-composants



- **En plus des composants de base « simples » décrits précédemment, Swing offre des macro-composants utiles, construits à partir des composants de base :**
  - **JColorChooser**, qui permet de choisir une couleur de différentes manières (via une palette, en choisissant ses composantes dans différents modèles, etc.),
  - **JFileChooser**, qui permet de choisir un ou plusieurs fichiers ou répertoires, existants ou non, en naviguant dans le système de fichiers.
- **Ces deux macro-composants peuvent s'utiliser comme des composants normaux ou dans des boîtes de dialogue.**



### 4.2.4.9 Composants de base : JOptionPane



- La classe **JOptionPane** offre une manière simple de créer des boîtes de dialogue modales en spécifiant un message, un titre, une icône, et un type de message ou un type d'option. Si aucune icône n'est spécifiée, des icônes sont fournies par le système en fonction du type de message.
- **JOptionPane** possède plusieurs **méthodes statiques** permettant d'ouvrir différents types de boîtes de dialogue:
  - Les boîtes d'information (**MessageDialog**) : Elles permettent d'afficher un message à l'utilisateur.
  - Les boîtes de saisie (**InputDialog**) : Elles invitent l'utilisateur à saisir du texte.
  - Les boîtes de confirmation (**ConfirmDialog**) : Elles demandent une confirmation à l'utilisateur.
  - Les boîtes personnalisées (**OptionDialog**) : Elles permettent d'inclure les trois autres types de boîtes.





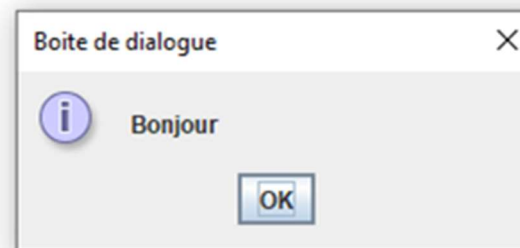
- La méthode **showMessageDialog** de la classe **JOptionPane** permet de faire apparaître un message à l'utilisateur :
  - showMessageDialog(Component c, Object message)
  - showMessageDialog(Component c, Object message, String title, int messageType)
  - showMessageDialog(Component c, Object message, String title, int messageType, Icon icon)
- Le composant **c** est le conteneur à considérer comme le parent de la boîte de dialogue. Si le paramètre **c** est null, la boîte de dialogue sera centrée sur l'écran.
- Le **messageType** peut être :
  - **JOptionPane.ERROR\_MESSAGE** : valeur entière 0.
  - **JOptionPane.INFORMATION\_MESSAGE** : valeur entière 1.
  - **JOptionPane.WARNING\_MESSAGE** : valeur entière 2
  - **JOptionPane.QUESTION\_MESSAGE** : valeur entière 3





- **JOptionPane.PLAIN\_MESSAGE**  
(**DEFAULT\_OPTION**) : valeur entière -1 pour afficher un message standard sans icône spécifique.

- La méthode **showMessageDialog** retourne **void**.



### 4.2.4.9 Composants de base : JOptionPane



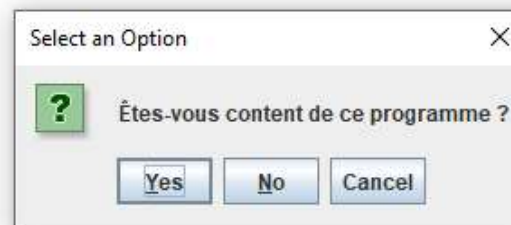
- La méthode **showConfirmDialog** de la classe **JOptionPane** affiche un message, avec une demande de confirmation :
  - `showConfirmDialog(Component c, Object message)`
  - `showConfirmDialog(Component c, Object message, String title, int optionType)`
  - `showConfirmDialog(Component c, Object message, String title, int optionType, int messageType)`
  - `showConfirmDialog(Component c, Object message, String title, int optionType, int messageType, Icon icon)`
- L'**optionType** peut être :
  - **JOptionPane.YES\_NO\_OPTION** : valeur entière 0.
  - **JOptionPane.YES\_NO\_CANCEL\_OPTION** : valeur entière 1.
  - **JOptionPane.OK\_CANCEL\_OPTION** : valeur entière 2
  - **JOptionPane.DEFAULT\_OPTION** (bouton de réponse OK seulement) : valeur entière -1.





● La méthode **showConfirmDialog** retourne un entier (**int**) qui peut avoir les valeurs suivantes en fonction des options choisies par l'utilisateur :

- **JOptionPane.YES\_OPTION** : valeur entière 0.
- **JOptionPane.NO\_OPTION** : valeur entière 1.
- **JOptionPane.CANCEL\_OPTION** : valeur entière 2.
- **JOptionPane.OK\_OPTION** : valeur entière 0.
- **JOptionPane.CLOSED\_OPTION** : valeur entière -1.



### 4.2.4.9 Composants de base : JOptionPane



#### ● La méthode **showInputDialog** de la classe **JOptionPane** permet de faire une saisie de chaîne de caractères :

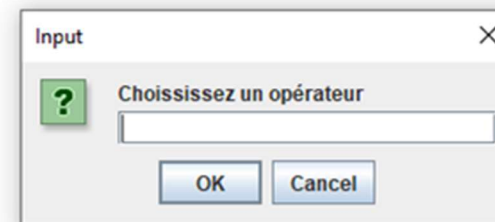
- `showInputDialog(Component c, Object message)` : Affiche une boîte de dialogue avec un message et un champ de saisie. Retourne la chaîne saisie par l'utilisateur ou null si la saisie est annulée.
- `showInputDialog(Object message)` : Variante de la précédente, mais utilisable sans spécifier le composant parent. Affiche une boîte de dialogue avec un message et un champ de saisie. Retourne la chaîne saisie par l'utilisateur ou null si la saisie est annulée.
- `showInputDialog(Component c, Object message, Object initV)` : Affiche une boîte de dialogue avec un message, un champ de saisie et une valeur initiale. Retourne la chaîne saisie par l'utilisateur ou null si la saisie est annulée.
- `showInputDialog(Object message, Object initV)` : Variante de la précédente, mais utilisable sans spécifier le composant parent. Affiche une boîte de dialogue avec un message, un champ de saisie et une valeur initiale. Retourne la chaîne saisie par l'utilisateur ou null si la saisie est annulée.



### 4.2.4.9 Composants de base : JOptionPane



- `showInputDialog(Component c, Object message, String title, int messageType)` : Affiche une boîte de dialogue avec un message, un titre et un type de message. Retourne la chaîne saisie par l'utilisateur ou null si la saisie est annulée.
  - `showInputDialog(Component c, Object message, String title, int messageType, Icon icon, Object[] Val, Object initV)` : Affiche une boîte de dialogue plus complexe avec un message, un titre, un type de message, une icône, une liste de valeurs possibles et une valeur initiale. Retourne un **objet** représentant la valeur sélectionnée par l'utilisateur dans la liste ou null si la saisie est annulée.
- Dans **showInputDialog**, si l'utilisateur ferme la boîte sans valider, null est retourné. L'**optionType** est toujours **OK\_CANCEL\_OPTION** et le **messageType** par défaut est **QUESTION\_MESSAGE**.





### 4.2.4.9 Composants de base : JOptionPane



- La méthode **showOptionDialog** de la classe `JOptionPane` est utilisée pour afficher une boîte de dialogue personnalisée avec des options spécifiques pour l'utilisateur. Elle permet de définir différents paramètres tels que le message, le titre, les options de boutons, l'icône, et une valeur initiale sélectionnée dans la boîte de dialogue. Cela offre une manière flexible d'interagir avec l'utilisateur en lui proposant des choix et des actions à effectuer en fonction du contexte.
  - `showOptionDialog(Component c, Object message, String title, int optionType, int messageType, Icon icon, Object[] options, Object initV)`
- Cette méthode retourne un entier (**int**) indiquant la position de l'option choisie par l'utilisateur parmi celles spécifiées dans le tableau `options`, ou `JOptionPane.CLOSED_OPTION` (valeur entière -1) si l'utilisateur a fermé cette boîte de dialogue.





● **Développer une calculatrice qui n'utilise pas d'interface graphique mais uniquement des boîtes de dialogue. Voici le déroulement de ce programme :**

1. Afficher un message de bienvenue dans une boîte de dialogue d'information avec le titre "Jeu".
2. Afficher une boîte de dialogue avec deux options "masculin" et "féminin" pour que l'utilisateur choisisse son sexe.
3. Afficher un message indiquant le sexe choisi par l'utilisateur.
4. Demander à l'utilisateur de saisir un premier nombre.
5. Demander à l'utilisateur de saisir un deuxième nombre.
6. Demander à l'utilisateur de choisir un opérateur parmi les options suivantes : +, -, \*, /. Si aucune option n'est sélectionnée, l'opération par défaut sera l'addition (+).
7. En fonction de l'opérateur choisi, effectuer le calcul correspondant et afficher le résultat dans une boîte de dialogue.
8. Demander à l'utilisateur s'il est satisfait du programme.
9. Afficher un message de sortie en fonction de la réponse.



- **Pour organiser les menus dans une interface Java, il est recommandé d'utiliser la classe **JMenuBar** pour afficher une barre de menus en haut de la fenêtre JFrame.**
  - Pour créer une barre de menus, il suffit d'instancier la classe **JMenuBar**, et d'ajouter ensuite la barre de menus au panneau (JFrame ou JPanel) .
  - Pour créer un menu déroulant et l'ajouter à la barre de menus, il suffit d'instancier la classe **JMenu**. Chaque menu peut contenir plusieurs items de menu (un item peut déclencher une action ou un événement de type ActionListener (voir plus loin) quand il est choisi). Il y a trois sortes d'item de menu :
    - **JMenuItem** : Cette classe permet de créer des items de menu simples. Ils peuvent comporter un texte, une icône, un raccourci clavier, un accélérateur, et peuvent être activés ou désactivés.
    - **JRadioButtonMenuItem** : Cette classe permet de créer des items de menu sous forme de boutons radio, ce qui est utile

### 4.2.4.10 Composants de base : Menu



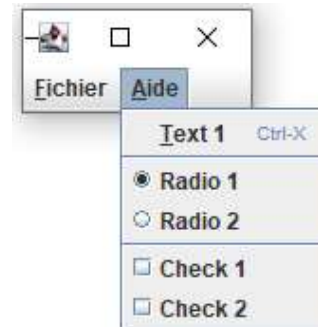
pour des choix exclusifs. L'utilisation d'un `ButtonGroup` permet de grouper ces boutons radio et garantit qu'un seul d'entre eux peut être sélectionné à la fois.

- **JCheckBoxMenuItem** : Cette classe permet de créer des items de menu sous forme de cases à cocher. Les méthodes `isSelected()` et `setSelected(boolean b)` permettent de vérifier si la case est cochée, et de la cocher ou de la décocher.
- Pour créer un menu surgissant, également appelé menu contextuel, qui apparaît sous forme de fenêtre contextuelle avec une série de choix à n'importe quel endroit dans un conteneur, il suffit d'instancier la classe **JPopupMenu**.
  - Un menu surgissant est rendu visible après un événement de la souris, tel que le relâchement du bouton droit sous Windows ou l'appui du bouton droit sous Linux. Pour afficher ce menu, il suffit d'utiliser la méthode `show(Component c, int x, int y)`, qui le positionne aux coordonnées (x, y) dans le composant c.





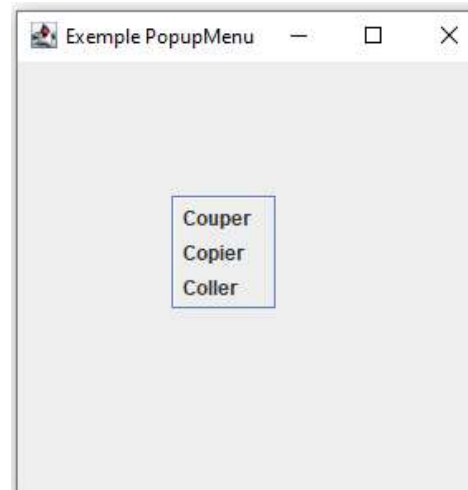
- **Créer un programme Java Swing qui affiche une fenêtre graphique avec une barre de menus. La barre de menus contient deux menus : "Fichier" et "Aide". Le menu "Fichier" est associé à la touche de raccourci clavier "F" et ne contient aucun élément. Le menu "Aide" est associé à la touche de raccourci "A" et contient plusieurs éléments :**
  - Un élément "Text 1" avec la touche de raccourci clavier "T" et un accélérateur "CTRL + X".
  - Deux boutons radio : "Radio 1" (sélectionné par défaut) et "Radio 2".
  - Deux cases à cocher : "Check 1" et "Check 2".





● **Développer une application Java Swing qui affiche un exemple de menu contextuel (JPopupMenu). L'application doit comporter les fonctionnalités suivantes :**

- Un menu contextuel comprenant les options "Couper", "Copier" et "Coller" doit s'afficher lorsque l'utilisateur clique avec le bouton droit de la souris dans la fenêtre.
- La fenêtre doit être dimensionnée à 300x300 pixels et doit être visible lors du lancement de l'application.





### 4.2.4.11 Composants de base : Composants ayant des modèles associés



- **Swing offre également des composants permettant d'afficher des données structurées : listes, tableaux, hiérarchie arborescente.**
  - Ces composants sont plus complexes à utiliser que les autres étant donné la nature des données qu'ils affichent.
  - Pour les obtenir, chaque composant complexe utilise un **modèle** qu'on lui fournit.



### 4.2.4.11 Composants de base : Composants ayant des modèles associés



- **Plusieurs composants de base permettent d'afficher et de manipuler des données complexes, par exemple :**
  - **JList** affiche une liste d'éléments,
  - **JTree** affiche une hiérarchie arborescente,
  - **JTable** affiche une table bidimensionnelle de cellules,
  - etc.
- **Où stocker les données affichées par ces composants et quel type leur attribuer ?**
  - Une solution serait de stocker les données directement dans les composants.
    - Par exemple, le composant **JList** pourrait stocker la liste des éléments qu'il affiche.
  - Cette solution n'est pas efficace car elle ne suit pas le patron de conception architectural **MVC** : les données à afficher font partie du modèle et ne doivent donc pas être stockée dans la vue.



### 4.2.4.11 Composants de base : Composants ayant des modèles associés

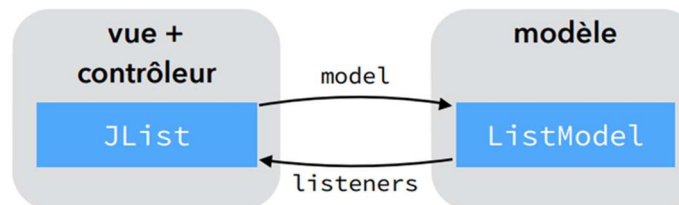


- Une meilleure solution est de faire en sorte que les composants puissent directement utiliser les données du modèle.
- **Pour ce faire, Swing offre la possibilité d'attacher à chacun des composants complexes précités un objet appelé modèle (model) représentant les données sous-jacentes.**
  - Ce modèle doit être observable au sens du **patron Observateur** pour permettre au composant de se mettre à jour automatiquement : le modèle est censé notifier la vue de toute modification.
  - Le type de ce modèle est spécifié par des interfaces fournies par Swing, et il est donc généralement nécessaire d'écrire un **adaptateur** pour adapter les données du programme aux types attendus par Swing.

### 4.2.4.11 Composants de base : Composants ayant des modèles associés



- Par exemple, le composant **JList** affiche une liste d'éléments et permet leur sélection. Il doit donc pouvoir effectuer les opérations suivantes sur la liste à afficher :
  - obtenir sa taille,
  - obtenir l'élément d'indice donné,
  - observer la liste pour être informé des changements.
- Ces opérations sont regroupées dans l'interface **ListModel**, représentant un modèle de liste.



- L'interface **ListModel** offre d'une part des méthodes permettant de gérer l'ensemble des observateurs — appelés ici listeners (ou auditeurs) — et d'autre part des méthodes de consultation de la liste de valeurs.



### 4.2.4.11 Composants de base : Composants ayant des modèles associés



```
public interface ListModel<E> {
 void addListDataListener(ListDataListener l);
 void removeListDataListener(ListDataListener l);

 int getSize();
 E getElementAt(int i);
}
```

- Pour faciliter la conception de modèles de listes, Swing propose la classe **AbstractListModel**. Cette classe implémente l'interface **ListModel** en gardant uniquement les méthodes **getSize()** et **getElementAt()** abstraites, et fournit des méthodes pour gérer les listeners. En particulier, elle offre des méthodes commençant par **fire...** qui informent ou notifient les listeners des changements dans le modèle, de manière similaire à la méthode **notifyObservers** du patron Observateur. De plus, la méthode **getListDataListeners()** de **AbstractListModel** renvoie un tableau de **ListDataListener[]** contenant tous les listeners de données de liste enregistrés sur ce modèle.



### 4.2.4.11 Composants de base : Composants ayant des modèles associés



- L'interface **ListDataListener** agit comme un observateur de modèle de liste, ou listener, pour suivre les modifications.

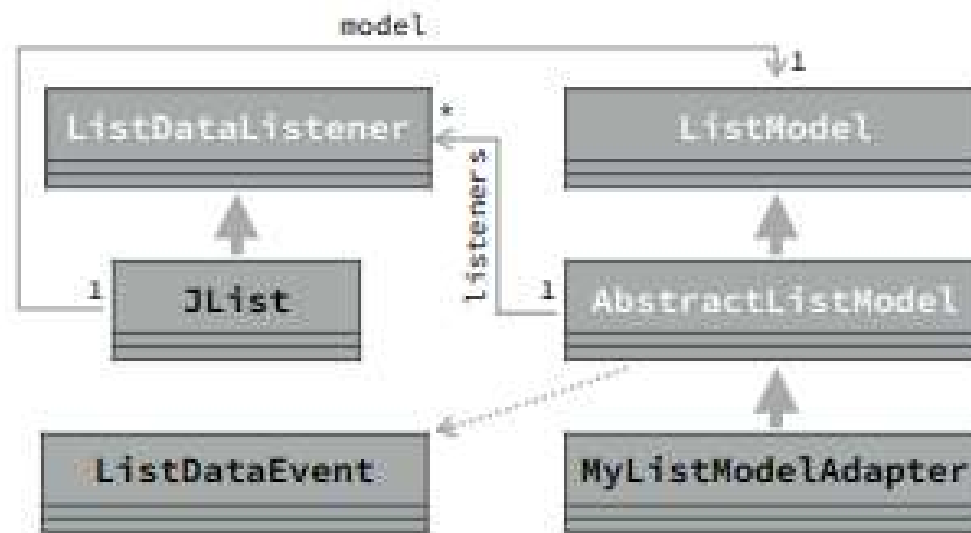
```
public interface ListDataListener {
 void intervalAdded(ListDataEvent e);
 void intervalRemoved(ListDataEvent e);
 void contentsChanged(ListDataEvent e);
}
```

- Cette interface offre trois méthodes de mise à jour, avec une généraliste (la troisième) et deux spécifiques pour les ajouts et les suppressions d'éléments.
- La classe **ListDataEvent** contient les informations liées au changement du contenu de la liste observée :
  - Le type de changement : ajout d'un intervalle, suppression d'un intervalle et changement général.
  - L'indice du premier élément affecté par le changement, accessible via la méthode `getIndex0()`.
  - L'indice du dernier élément affecté par le changement accessible via la méthode `getIndex1()`.



### 4.2.4.11 Composants de base : Composants ayant des modèles associés

- Le diagramme de classes ci-dessous illustre — de manière légèrement simplifiée — les classes qui seraient utilisées par une application incluant un composant **JList**.



### 4.2.4.11 Composants de base : Composants ayant des modèles associés

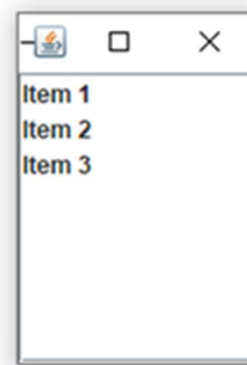


- La classe concrète **DefaultListModel**, héritant de **AbstractListModel**, permet d'ajouter et de supprimer des éléments dans une liste. Elle fournit des méthodes qui facilitent la gestion des éléments dans le modèle de liste telles que `add(int index, E elt)`, `addElement(E elt)`, `insertElementAt(E elt, int index)`, `remove(int index)`, `removeElement(Object o)`, `removeElementAt(int index)`, `get(int index)`, `firstElement()`, `lastElement()`, `indexOf(Object elt)` `set(int index, E elt)`, `clear()`, etc.
- Pour des données simples, sans avoir besoin de personnaliser la façon dont les éléments sont stockés ou affichés, **DefaultListModel** est une bonne option car elle offre des méthodes pratiques pour gérer les éléments dans une liste. Pour des exigences plus complexes où il faut personnaliser le stockage ou l'affichage des éléments, **AbstractListModel** est une meilleure option car elle permet de personnaliser davantage le comportement du modèle de liste selon les besoins spécifiques.

### 4.2.4.11 Composants de base : Composants ayant des modèles associés



- Créer une fenêtre Swing avec une liste déroulante (JList) avec trois éléments ("Item 1", "Item 2", "Item 3"). Ces éléments proviendront d'un modèle de liste personnalisé qui fournira les données à afficher dans la liste. Ensuite, placer cette liste dans un panneau déroulant (JScrollPane) pour permettre le défilement si nécessaire.
- Proposer une autre solution utilisant un modèle par défaut pour les éléments de la liste.





### 4.2.4.12 Composants de base : Composants personnalisés



- Lorsqu'une application nécessite un composant ne figurant pas parmi les prédéfinis (ceux déjà cités), il est possible de définir un composant personnalisé sous la forme d'une nouvelle sous-classe de JComponent.
- Les principales méthodes à redéfinir dans ce cas sont :
  - **getPreferredSize**, **getMinimumSize** et **getMaximumSize**, qui permettent aux gestionnaires d'agencement de le dimensionner correctement,
  - **paintComponent**, qui dessine le composant.
- Les méthodes **getPreferredSize**, **getMinimumSize** et **getMaximumSize** sont utilisées par certains gestionnaires de mise en page - mais pas tous - pour déterminer la taille du composant.
  - Il peut donc être utile de les redéfinir lorsque la taille du composant ne peut pas être arbitraire.



### 4.2.4.12 Composants de base : Composants personnalisés



● La méthode `paintComponent` est automatiquement appelée par Swing chaque fois que le composant doit être (re) dessiné, par exemple lorsqu'il apparaît pour la première fois à l'écran.

- Cette méthode reçoit en argument le contexte graphique à utiliser pour effectuer le dessin. Le repère qui lui est associé est celui du composant.
- Par exemple, le composant ci-dessous affiche un rectangle rouge centré sur un fond noir :

```
public final class RonB extends JComponent {
 protected void paintComponent(Graphics g) {
 int w = (int)getBounds().getWidth();
 int h = (int)getBounds().getHeight();
 g.setColor(Color.BLACK);
 g.fillRect(0, 0, w, h);
 g.setColor(Color.RED);
 g.fillRect(w/4, h/4, w/2, h/2);
 }
}
```

### 4.2.4.12 Composants de base : Composants personnalisés



- **Il peut arriver qu'un composant doit être redessiné, p.ex. à la suite d'un changement des données du modèle qu'il représente.**
  - Cela ne doit, en aucun cas, être fait en appelant directement `paintComponent` : cette méthode étant uniquement destinée à être appelée par Swing.
  - Au lieu de cela, il convient d'appeler la méthode **repaint** du composant, afin de demander à Swing de le redessiner (via sa méthode `paintComponent`) aussi vite que possible.
- **A noter que Swing redessine automatiquement les composants dont la taille a changé, sans qu'un appel à `repaint` ne soit nécessaire.**





- **Un programme doté d'une interface graphique ne fait généralement qu'attendre que l'utilisateur interagisse avec celle-ci, réagit en conséquence, puis se remet à attendre.**
  - Chaque fois que le programme est forcé de réagir, on dit qu'un événement (**event**) s'est produit.
- **Cette caractéristique des programmes graphiques induit à la programmation événementielle.**
- **Au cœur de tout programme événementiel se trouve une boucle traitant successivement les événements dans leur ordre d'arrivée.**
  - Cette boucle, nommée **boucle événementielle**, pourrait s'exprimer ainsi en pseudo-code :  
**tant que le programme n'est pas terminé**  
**attendre le prochain événement**  
**traiter cet événement**





- **Cette boucle est souvent fournie par une bibliothèque et ne doit dans ce cas pas être écrite explicitement. Il en va ainsi de la plupart des bibliothèques de gestion d'interfaces graphiques, dont Swing.**
  - Si la boucle événementielle peut être identique pour tous les programmes, la manière dont les différents événements sont gérés est bien entendu spécifique à chaque programme.
- **Dès lors, il doit être possible de définir la manière de traiter chaque événement. Cela se fait généralement en écrivant, pour chaque événement important, un morceau de code le gérant, appelé le gestionnaire d'événement.**
  - Ce gestionnaire est associé, d'une manière ou d'une autre, à l'événement.
- **La boucle événementielle est séquentielle, dans le sens où un événement ne peut être traité que lorsque le traitement de l'événement précédent est terminé.**



## 4- Swing

### 4.3 Gestion des événements



- Dans le cas d'une interface graphique, cela signifie que celle-ci est totalement bloquée tant et aussi longtemps qu'un événement est en cours de traitement.
- Pour cette raison, les **gestionnaires d'événements** doivent autant que possible **s'exécuter rapidement**.
  - Si cela n'est pas possible, il faut utiliser la concurrence pour effectuer les longs traitements en tâche de fond - ce qui sort du cadre de ce cours.



### 4.3.1 Gestion des événements dans Swing



- **Dans une application graphique Swing, la boucle événementielle s'exécute dans un fil d'exécution (thread) séparé, nommé le **fil de gestion des événements** (Event Dispatching Thread ou EDT).**
  - Ce fil d'exécution est démarré automatiquement lors de l'utilisation de Swing, rendant la boucle événementielle particulièrement invisible au programmeur.
- **Généralement, tous les appels à des méthodes de Swing doit se faire depuis son propre EDT. En particulier, toute la création de l'interface doit s'y faire.**
- **Une application graphique Swing peut comporter deux threads au minimum.**
  - Un thread pour le programme en lui-même (celui qui exécute le code à partir du main) et un thread pour Swing. La règle est simple, on ne fait rien de graphique dans le thread principal et on ne fait rien d'autre que du graphique dans le thread Swing.

### 4.3.1 Gestion des événements dans Swing



- Dans Swing, les gestionnaires d'événements sont des objets, appelés **listeners** (ou auditeurs), implémentant une interface qui définit une ou plusieurs méthodes de gestion d'événement.
  - Les listeners sont attachés à une source d'événements - généralement un composant - et leurs méthodes sont appelées chaque fois qu'un événement se produit.
- La classe **TestSwing**, ci-après, illustre la structure typique d'un programme Swing.
  - L'interface graphique est créée par la méthode **createUI**, appelée indirectement via **invokeLater** afin de garantir qu'elle s'exécute sur l'EDT.
    - La méthode **createUI** crée une fenêtre et y ajoute un bouton dont le listener affiche un message "click!" à l'écran lorsqu'on lui clique dessus.





```
import javax.swing.*;

public class TestSwing {
 private static void createUI() {
 JFrame w = new JFrame();
 w.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 JButton b = new JButton("click me!");
 b.addActionListener(e -> System.out.println("click!"));
 w.getContentPane().add(b);
 w.pack();
 w.setVisible(true);
 }

 public static void main(String[] args) {
 SwingUtilities.invokeLater(() -> createUI());
 }
}
```

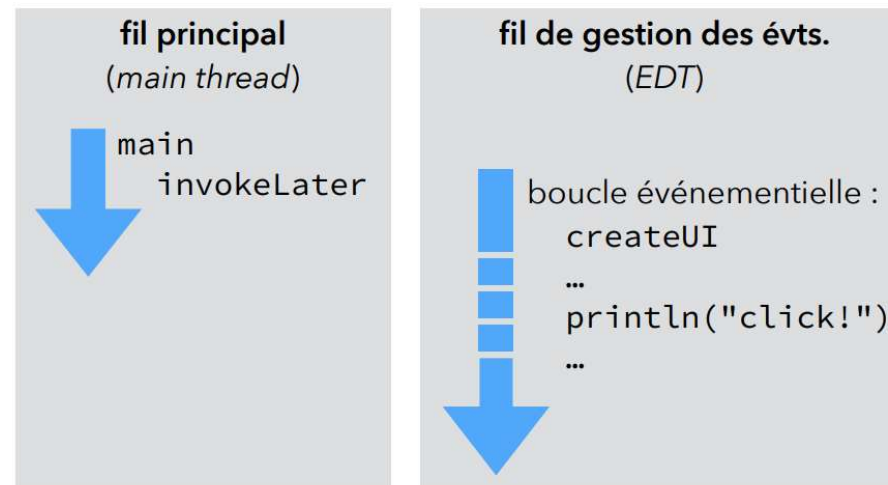




### 4.3.1 Gestion des événements dans Swing



- La figure, ci-dessous, illustre l'exécution des différentes parties du programme sur les deux fils existants.



- Lorsqu'un événement se produit, Swing collecte les informations à son sujet dans un objet passé au listener.
  - Par abus de langage, cet objet lui-même est nommé événement (event).
- Généralement, à chaque type d'événement correspond un type de listener et un type d'objet-événement.



- **Swing définit plusieurs types de listeners, en fonction du type d'événement qu'ils écoutent. Les principaux types de listeners sont :**
  - les auditeurs d'action (**action listeners**),
  - les auditeurs de souris (**mouse listeners**),
  - les auditeurs de menu (**menu listeners**),
  - les auditeurs de clavier (**key listeners**),
  - les auditeurs de cible de saisie (**focus listeners**).
- **Seuls les deux premiers listeners seront examinés dans ce cours, les autres ayant un fonctionnement similaire.**





- L'interface fonctionnelle **ActionListener** représente un listener à l'écoute des événements de type « action ».
    - Ces événements se produisent p.ex. lorsque l'utilisateur clique sur un bouton, sélectionne un menu ou presse Entrée dans un champ textuel.
- ```
public interface ActionListener {  
    void actionPerformed(ActionEvent e);  
}
```
- La classe **ActionEvent** contient différentes informations au sujet de l'événement, p.ex. les touches de modifications (Control, Shift, ...) pressées au moment du déclenchement de l'action.
 - Un listener de type **ActionListener** peut être attaché à plusieurs types de composants, principalement les boutons, les champs textuels et les entrées de menus. Chacun de ces types de composants offre les méthodes **addActionListener** et **removeActionListener** pour ajouter/supprimer un auditeur d'action.





- L'interface **MouseListener** représente un listener à l'écoute de certains événements liés à la souris : entrée et sortie des bornes du composant, pression et relâchement des boutons :

```
public interface MouseListener {  
    void mouseEntered(MouseEvent e);  
    void mouseExited(MouseEvent e);  
    void mouseClicked(MouseEvent e);  
    void mousePressed(MouseEvent e);  
    void mouseReleased(MouseEvent e);  
}
```

- La classe **MouseEvent** contient différentes informations concernant l'événement, p.ex. la position de la souris.





- L'interface **MouseMotionListener** représente un listener à l'écoute des événements liés au déplacement de la souris, avec un bouton enfoncé (**mouseDragged**) ou relâché (**mouseMoved**) :

```
public interface MouseMotionListener {  
    void mouseDragged(MouseEvent e);  
    void mouseMoved(MouseEvent e);  
}
```

- Chaque mouvement de la souris constituant un nouvel événement, ceux-ci peuvent être très nombreux.
- Les listeners de type **MouseMotionListener** se doivent donc de traiter rapidement les événements.





- L'interface **MouseWheelListener** représente un listener à l'écoute des événements liés à la molette de la souris :

```
public interface MouseWheelListener {  
    void mouseWheelMoved(MouseWheelEvent e);  
}
```

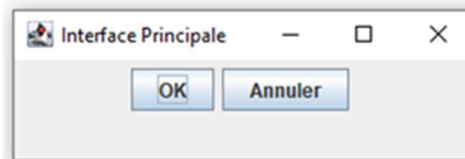
- La classe **MouseWheelEvent** étend **MouseEvent** en lui ajoutant des informations liées à la molette : distance et sens de rotation, etc.
- Les trois types de listeners liés à la souris peuvent être attachés à (et détachés de) n'importe quel composant au moyen des méthodes **addMouse...Listener** et **removeMouse...Listener**.
- La classe abstraite **MouseAdapter** fournit une mise en œuvre par défaut de ces trois interfaces **Mouse...Listener**
 - Elle implémente toutes les méthodes de ces trois interfaces, en ne faisant rien dans tous les cas.



4.4 Exemples d'interfaces graphiques avec Swing



- Créer une fenêtre Swing avec deux boutons (OK et Annuler) et un JLabel. Lorsque on clique sur le bouton "OK", le texte du JLabel est défini sur "OK", et lorsque on clique sur le bouton "Annuler", le texte du JLabel est défini sur "Annuler". Gérer également les événements de la souris en affichant un message sur la console chaque fois qu'un clic de souris sur le bouton OK est détecté.



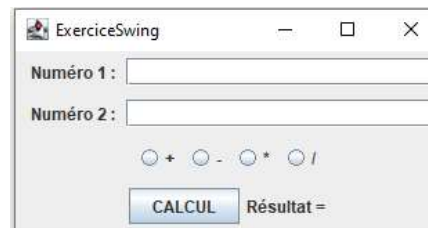
4.4 Exemples d'interfaces graphiques avec Swing



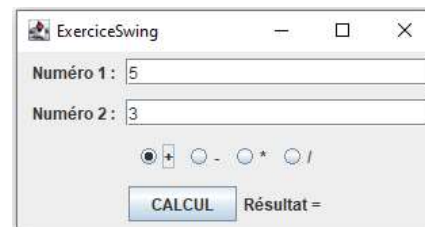
- **Écrire un programme qui permet à l'utilisateur de saisir deux nombres entiers dans deux champs de texte (JTextField) et qui affiche le résultat de l'opération (selon le bouton radio sélectionné) sur ces deux nombres lorsque l'utilisateur clique sur le bouton CALCUL.**
 - Le programme devra gérer convenablement le cas où l'utilisateur clique sur le bouton CALCUL après avoir saisi une autre chose qu'un nombre entier dans un champ de texte ; le programme remettra, dans ce cas, les champs de texte à blanc et affichera un message d'erreur sur la console.
- **Des scénarios d'exécution de cette application sont présentés dans le tableau suivant :**



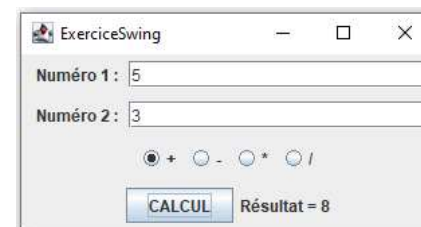
4.4 Exemples d'interfaces graphiques avec Swing



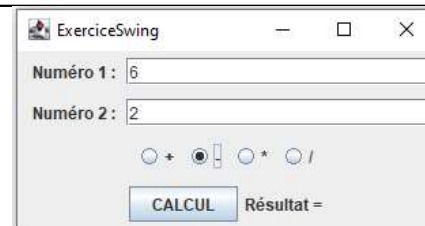
(a) Aperçu de l'interface graphique lors du démarrage de l'application



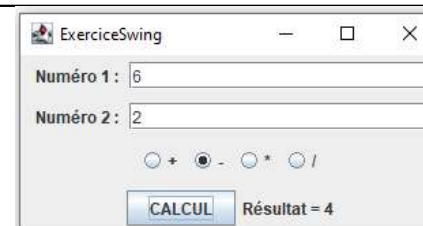
(b1) Aperçu de l'interface après avoir saisi deux entiers et coché l'opération (+)



(b2) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL

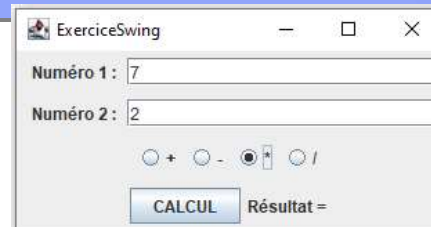


(c1) Aperçu de l'interface après avoir saisi deux entiers et coché l'opération (-)

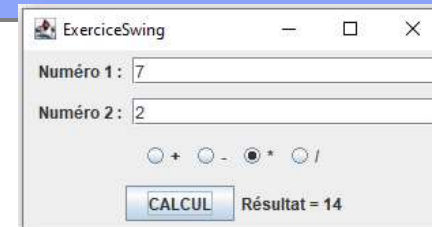


(c2) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL

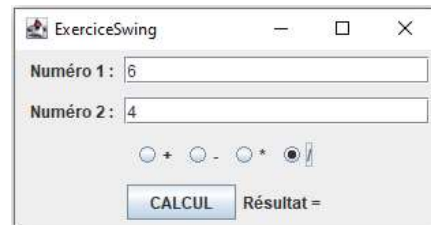
4.4 Exemples d'interfaces graphiques avec Swing



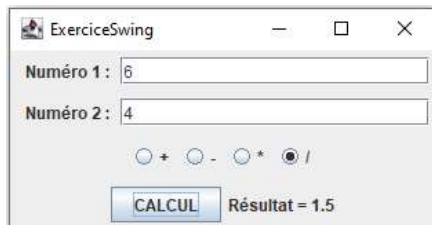
(d1) Aperçu de l'interface après avoir saisi deux entiers et coché l'opération (*)



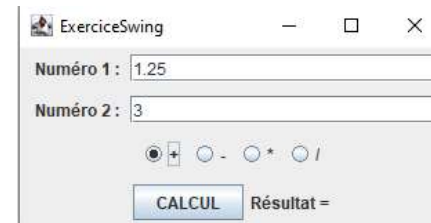
(d2) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL



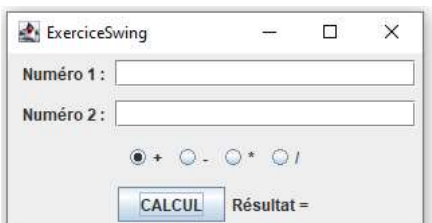
(e1) Aperçu de l'interface après avoir saisi deux entiers et coché l'opération (/)



(e2) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL



(f1) Aperçu de l'interface après avoir saisi un numéro non entier (1.25) et coché l'opération (+) par exemple



(f2) Aperçu de l'interface et de la console après avoir cliqué sur le bouton CALCUL

